

Proiectarea cu Microprocesoare

Microsistem cu microprocesorul 8086

Stochean Liviana

2025-2026

3.2C

Tema proiectului:

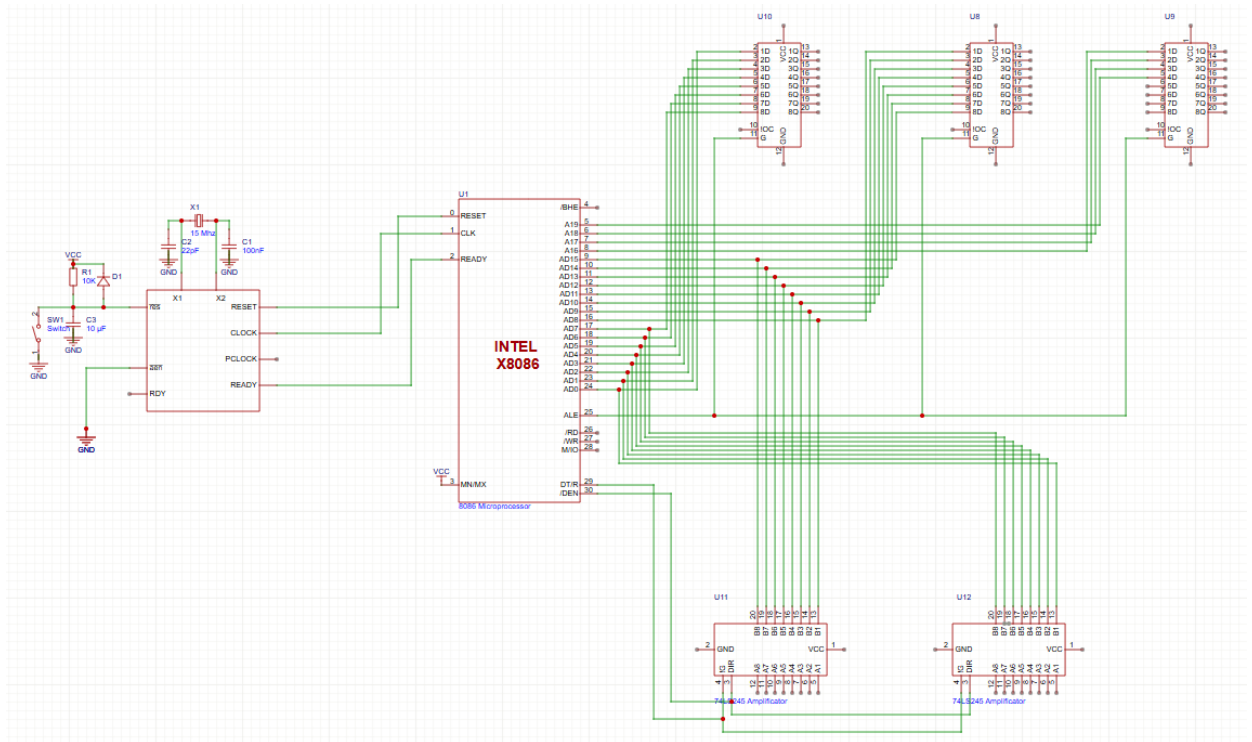
Să se proiecteze un microsistem cu următoarea structură:

- unitate centrală cu microprocesorul 8086;
- 128 KB memorie EPROM, utilizând circuite 27C2048;
- 64 KB memorie SRAM, utilizând circuite 62512;
- interfață serială, cu circuitul 8251, plasată în zona 0AF0H – 0AF2H sau 0BF0H – 0BF2H, în funcție de poziția microcomutatorului S1;
- interfață paralelă, cu circuitul 8255, plasată în zona 0D70H – 0D76H sau 0C70H – 0C76H, în funcție de poziția microcomutatorului S2;
- o minitastatură cu 12 contacte;
- 6 led-uri;
- un modul de afișare cu 7 segmente, cu 6 ranguri;

Toate programele în limbaj de asamblare vor fi concepute sub formă de subrutine. Programele necesare sunt:

- rutinele de programare ale circuitelor 8251 și 8255;
- rutinele de emisie/ recepție caracter pe interfața serială;
- rutina de emisie caracter pe interfață paralelă;
- rutina de scanare a minitastaturii;
- rutina de aprindere/ stingere a unui led;
- rutina de afișare a unui caracter hexa pe un rang cu segmente.

1. Unitatea centrală

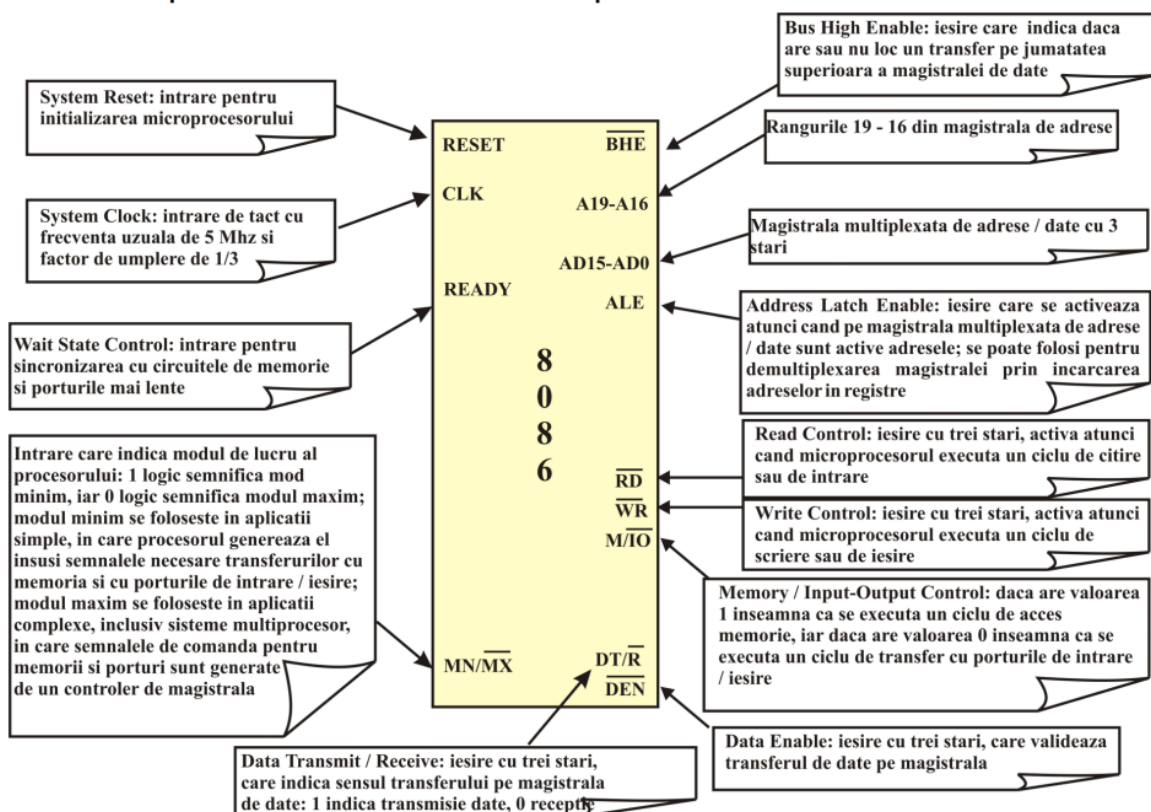
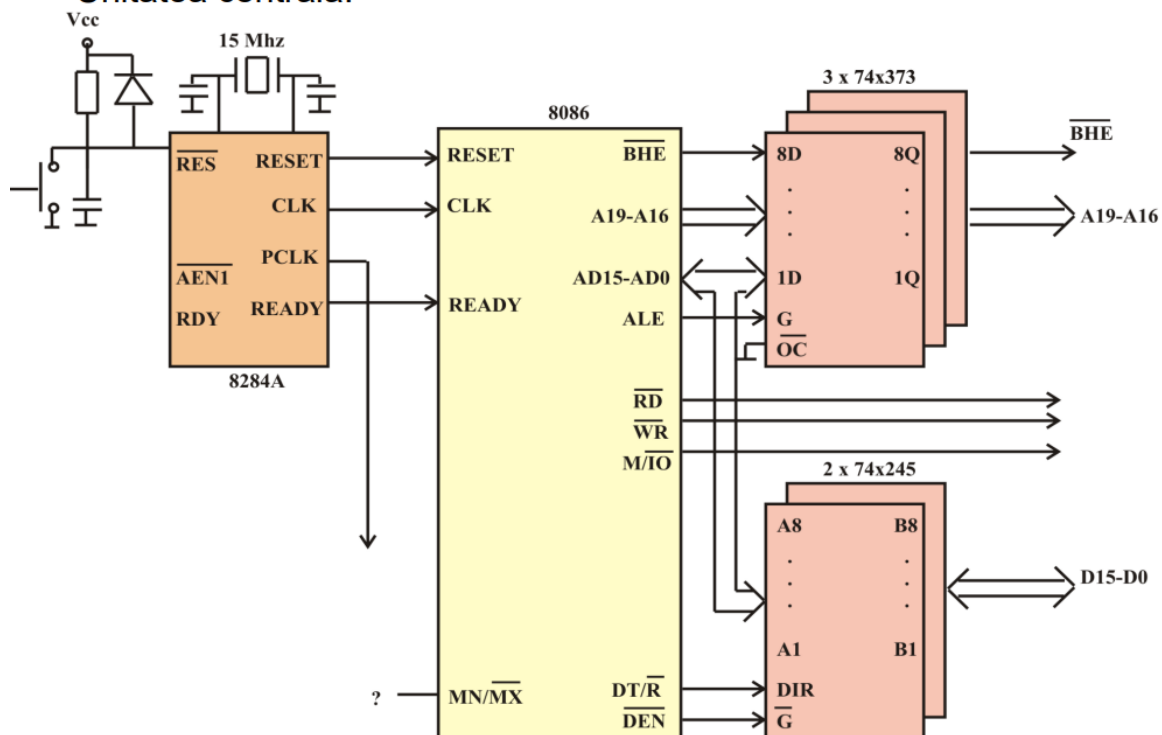


Microprocesorul 8086 și rolul său în sistem

Microprocesorul **Intel 8086** reprezintă elementul central al microsistemului și are rolul de a coordona toate operațiile de prelucrare și control. Acesta execută instrucțiunile programului, gestionează transferul de date și controlează accesul la memorie și la dispozitivele periferice.

8086 este un procesor pe **16 biți**, având o magistrală de date pe 16 biți și o magistrală de adrese pe 20 de biți, ceea ce permite adresarea unui spațiu de memorie de până la **1 MB**. În proiectul de față, microprocesorul funcționează în **Minimum Mode**, generând direct semnalele de control necesare sistemului.

● Unitatea centrală:



Magistrale și organizarea liniilor de adresă și date

Una dintre caracteristicile importante ale microprocesorului 8086 este utilizarea **liniilor multiplexate adresă/date**. Liniile **AD0–AD15** sunt folosite inițial pentru transmiterea părții inferioare a adresei, iar ulterior, în același ciclu de magistrală, sunt reutilizate pentru transferul de date.

Pentru ca adresa să rămână stabilă pe toată durata operației, este necesară separarea adreselor de date. Acest lucru este realizat cu ajutorul **latch-urilor 74LS373**, controlate de semnalul **ALE (Address Latch Enable)**.

Funcționarea este următoarea:

- la începutul ciclului de magistrală, pe liniile AD0–AD15 apare adresa;
- semnalul **ALE** devine activ și determină memorarea adresei în latch-uri;
- după dezactivarea ALE, ieșirile latch-urilor păstrează adresa stabilă;
- liniile AD0–AD15 pot fi reutilizate pentru transferul datelor.

Liniile **A16–A19** completează magistrala de adrese și sunt multiplexate cu semnale de stare, indicând tipul operației în desfășurare.

Magistrala de date și transceiverele 74LS245

Transferul efectiv al datelor se realizează prin magistrala **D0–D15**, care este bidirecțională. Pentru a separa și proteja această magistrală, sunt utilizate **transceiverele 74LS245**.

Rolul acestora este:

- amplificarea semnalelor de date;
- izolarea procesorului de restul sistemului atunci când nu are loc un transfer;
- controlul direcției de transfer a datelor.

Direcția transferului este stabilită de semnalul **DT/ $\bar{R}$** , iar activarea transceiverelor este realizată prin semnalul **DE $\bar{N}$** .

Semnale de control ale microprocesorului

Microprocesorul 8086 generează o serie de semnale de control esențiale pentru funcționarea sistemului:

- **$\overline{RD}$ (Read)** – comandă citirea datelor din memorie sau din periferice;
- **$\overline{WR}$ (Write)** – comandă scrierea datelor către memorie sau periferice;
- **$\overline{M/IO}$** – diferențiază accesul la memorie de accesul la spațiul de intrare/ieșire;
- **$\overline{BHE}$ (Bus High Enable)** – indică utilizarea octetului superior al magistralei de date și permite accesul pe 8 sau 16 biți;
- **$A0$** – selectează octetul inferior în transferurile pe 8 biți.

Prin combinarea semnalelor **$A0$** și **$\overline{BHE}$** , procesorul poate accesa:

- doar octetul inferior;
- doar octetul superior;
- ambii octeți simultan, în cazul transferurilor pe 16 biți.

Semnale de sincronizare și control al execuției

Funcționarea procesorului este sincronizată cu ajutorul semnalului **CLK**, furnizat de circuitul **8284A**, care utilizează un cristal de cuarț de **15 MHz**. Acest semnal stabilește ritmul tuturor operațiilor interne și externe ale sistemului.

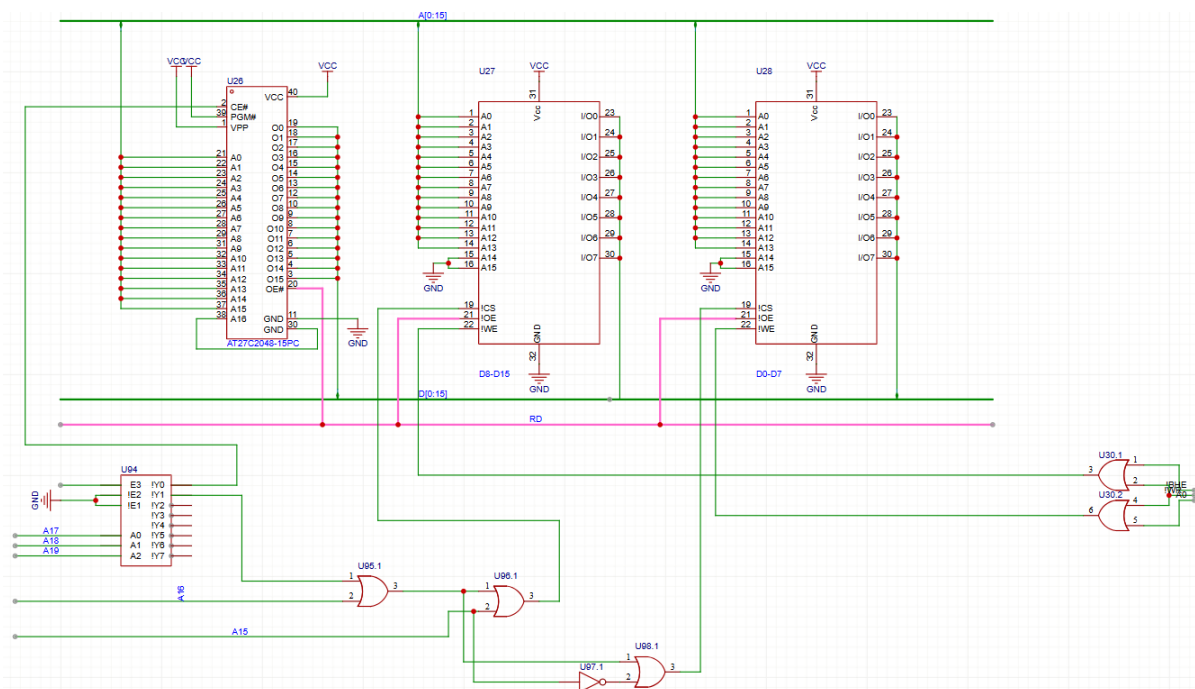
Semnalul **RESET** readuce procesorul într-o stare inițială cunoscută la pornirea sistemului, iar semnalul **READY** permite introducerea unor stări de așteptare atunci când memoria sau perifericele sunt mai lente decât procesorul.

Funcționarea generală a unității centrale

În funcționarea normală, microprocesorul generează adresa, o stabilizează cu ajutorul latch-urilor, selectează memoria sau perifericul corespunzător și controlează transferul de date prin semnalele de citire sau scriere. Toate aceste operații sunt sincronizate cu semnalul de ceas și coordonate de unitatea centrală.

Prin utilizarea circuitelor auxiliare (8284A, 74LS373, 74LS245), unitatea centrală asigură o funcționare corectă și stabilă a microsistemului, permițând comunicarea eficientă între procesor, memorie și periferice.

2. Memorii



➔ Memoria EPROM – definiție, pini și semnale (27C2048)

Memoria EPROM 27C2048 este utilizată în sistem pentru stocarea programului executat de microprocesor. Este o memorie nevolatilă, ceea ce înseamnă că informațiile stocate se păstrează chiar și după întreruperea alimentării. În cadrul proiectului, EPROM-ul este folosit exclusiv pentru operații de citire, conținutul său fiind stabilit anterior funcționării sistemului.

Circuitul EPROM este organizat intern pe locații adresabile, selecția acestora realizându-se prin intermediul liniilor de adresă A0–A16. În configurația utilizată, memoria ocupă un spațiu continuu de 128 KB, mapat între adresele 00000h și 1FFFFh.

Pini și semnale importante ale EPROM-ului:

- A0–A16 (Address Lines)
determină locația de memorie accesată în interiorul EPROM-ului; fiecare combinație corespunde unei locații distincte.
- O0–O7 (Data Outputs)
reprezintă ieșirile de date ale memoriei, conectate la magistrala de date a sistemului; datele sunt puse pe magistrală doar în timpul unei operații de citire.
- $\overline{CE}$ (Chip Enable)
activează circuitul EPROM; când acest semnal este activ (nivel LOW), memoria răspunde la acces.
- $\overline{OE}$ (Output Enable)
permite plasarea datelor pe ieșiri; este conectat la semnalul $\overline{RD}$ al microprocesorului.
- VPP / $\overline{PGM}$
pini utilizați doar în timpul programării EPROM-ului; în regim normal de funcționare sunt legați la VCC.
- VCC / GND
alimentarea circuitului.

În schema prezentată, semnalul $\overline{CE}$ al EPROM-ului este furnizat de decodificatorul de adrese 74LS138, astfel încât memoria este activă doar atunci când procesorul accesează intervalul de adrese rezervat acesteia.

➔ *Memoria SRAM – definiție, pini și semnale (62512)*

Memoria SRAM 62512 este utilizată ca memorie de date a sistemului. Spre deosebire de EPROM, SRAM-ul este o memorie volatilă, informațiile fiind pierdute la întreruperea alimentării. Avantajul principal constă în viteza mare de acces, ceea ce o face potrivită pentru stocarea variabilelor și rezultatelor intermediare.

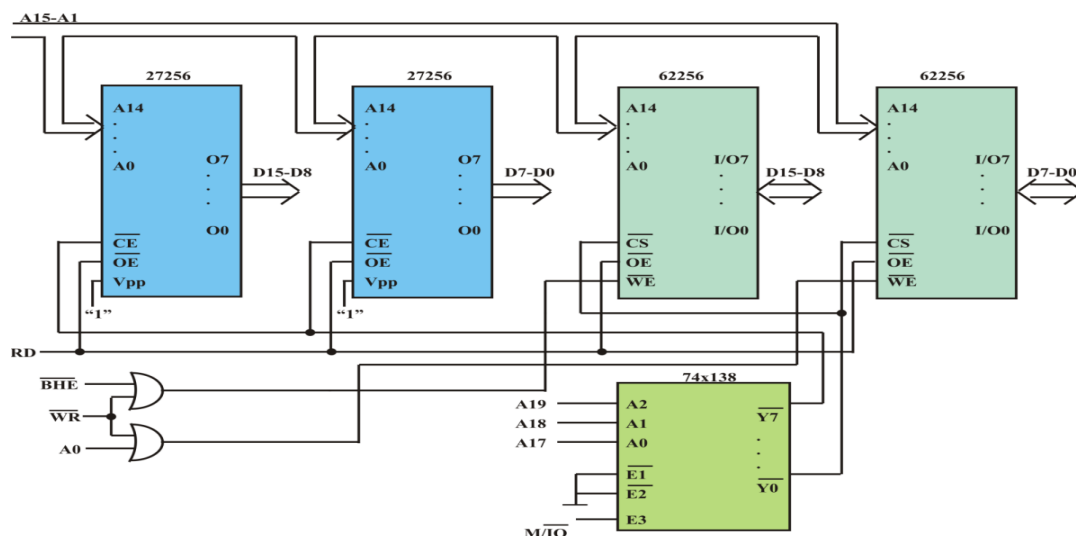
În proiect sunt utilizate două circuite SRAM, fiecare organizat pe 8 biți, care împreună formează o memorie de 16 biți, compatibilă cu magistrala de date

a microprocesorului 8086. Fiecare circuit este utilizat pentru câte un octet (low și high byte).

Pini și semnale importante ale SRAM-ului:

- A0–A15 (Address Lines)
selectează locația de memorie internă accesată.
- I/O0–I/O7 (Data Lines)
linii bidirecționale, utilizate atât pentru citirea, cât și pentru scrierea datelor.
- $\overline{CS}$ (Chip Select)
activează circuitul SRAM; semnalul este generat de logica de decodificare a adreselor.
- $\overline{OE}$ (Output Enable)
permite transferul datelor din memorie către magistrală în timpul operațiilor de citire; este conectat la $\overline{RD}$.
- $\overline{WE}$ (Write Enable)
controlează scrierea datelor în memorie; este conectat la $\overline{WR}$ al procesorului.
- VCC / GND
alimentarea circuitului.

Cele două circuite SRAM sunt accesate simultan în timpul operațiilor pe 16 biți, fiecare furnizând câte un octet de date (inferior și superior).



Memory Mapping

→ EPROM 128KB

Se cere proiectarea unui microsistem cu memorie **EPROM de 128 KB**, utilizând circuite de tip **27C2048**. Circuitul EPROM 27C2048 are o capacitate totală de **2 Mb**, ceea ce corespunde unei memorii de **256 KB**. Având în vedere cerințele proiectului, nu este necesară utilizarea întregii capacități a circuitului.

Pentru obținerea unei memorii de **128 KB**, se utilizează doar jumătate din spațiul de adresare al EPROM-ului. În acest scop, linia de adresă superioară **A16** este conectată la nivel logic zero (GND), limitând astfel accesul la o singură jumătate a memoriei interne. Prin această metodă, circuitul EPROM este împărțit logic în două zone egale, fiind utilizată doar una dintre acestea.

Dimensiunea de 128 KB corespunde unui număr de **2^{17} locații adresabile**, fiind necesare liniile de adresă **A0–A16**. Memoria EPROM este mapată într-un spațiu continuu de adrese, începând de la adresa **00000h**, ceea ce permite microprocesorului să acceseze direct programul de pornire și instrucțiunile stocate.

Selecția EPROM-ului se realizează prin intermediul unui circuit de decodificare a adreselor, care analizează liniile superioare ale magistralei de adrese și generează semnalul de activare al memoriei doar pentru intervalul de adrese rezervat acesteia. Astfel, se asigură o separare clară între memoria de program și celelalte zone de memorie ale sistemului.

---EPROM 27C2048:

$$2\text{Mb} = 128\text{KB} * 16\text{b}(\text{word length}) = 128\text{KB} * 2\text{B} = 256\text{KB}.$$

Doar că nouă în cerință ne cere de 128KB așa că am conectat A16 la GND, pentru a nu folosi întreaga capacitate a circuitului.

$128\text{KB} = 2^7 * 2^{10} = 2^{17}\text{B}$, asta însemnând că dimensiunea aceasta acoperă 20000H locații ceea ce ne ajută la tabel.

➔ SRAM 64KB

Pentru stocarea datelor temporare, microsistemul utilizează o memorie **SRAM de 64 KB**, realizată cu ajutorul a două circuite de tip **62512**. Fiecare circuit SRAM 62512 este organizat ca **64K × 8 biți**, fiind destinat stocării unui singur octet de date.

Având în vedere faptul că microprocesorul 8086 dispune de o magistrală de date pe **16 biți**, utilizarea a două circuite SRAM permite realizarea unei memorii de date compatibile cu această arhitectură. Unul dintre circuite este utilizat pentru transferul **octetului inferior (D0–D7)**, iar celălalt pentru transferul **octetului superior (D8–D15)**.

În cadrul proiectului nu este necesară utilizarea întregii capacități a fiecărui circuit SRAM. Din acest motiv, fiecare memorie este utilizată pe o capacitate de **32 KB**, rezultând împreună o memorie de **64 KB × 16 biți**. Limitarea capacității se realizează prin selecția corespunzătoare a liniilor de adresă, utilizând logica de decodificare a adreselor.

Zona de memorie RAM este mapată imediat după memoria EPROM, ocupând un spațiu continuu în harta de memorie a sistemului. Pentru o organizare clară, aceasta este împărțită în două segmente de câte **32 KB**, fiecare fiind selectat printr-un semnal distinct de activare generat de circuitul de decodificare.

Selecția circuitelor SRAM este realizată prin intermediul decodicatorului de adrese, care analizează liniile superioare ale magistralei de adrese și activează memoria RAM doar atunci când procesorul accesează intervalul de adrese rezervat acestuia. Semnalele de control ale memoriei SRAM sunt conectate direct la semnalele de citire și scriere ale microprocesorului, permițând atât operații de citire, cât și de scriere a datelor.

Această soluție asigură o memorie de date rapidă și eficientă, necesară funcționării corecte a microsistemului, și este pe deplin compatibilă cu arhitectura pe 16 biți a microprocesorului 8086.

---SRAM 64KB

64KB * 8b(word length), dar avem nevoie de 64KB * 16b, așa că vom împărți circuitul în 2 SRAM de tip 62512 și legăm A14 și A15 la GND pentru a obține 2 circuite de 32KB.

$32KB = 2^5 * 2^{10} = 2^{15}B$ ce acoperă 8000H locații.

Tabele:

	Adresa de start	Adresa de stop
EPROM 1	00000H	1FFFFH
SRAM 1	20000H	27FFFH
SRAM 2	28000H	2FFFFH

Stop=start+dimensiune-1

Eprom 1: $00000H + 20000H - 1 = 1FFFFH$

Sram 1: $20000H + 8000H - 1 = 27FFFH$

Sram 2: $28000H + 8000H - 1 = 2FFFFH$

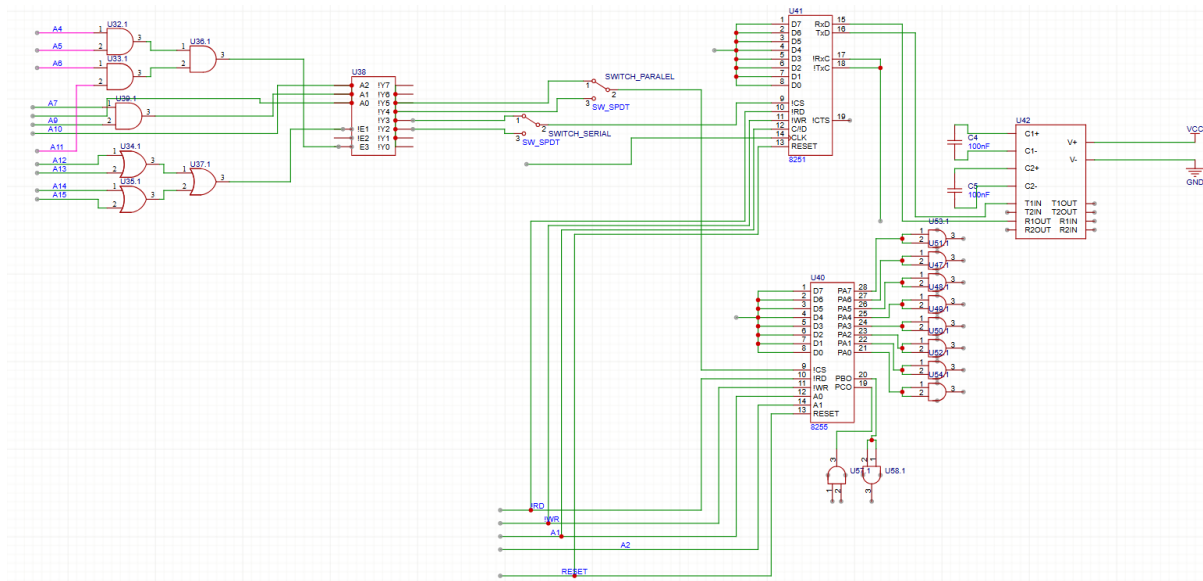
Circuitul	A19	A18	A17	A16	A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0
Eprom start	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Eprom stop	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Sram 1 Start	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Sram 1 Stop	0	0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Sram 2 Start	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Sram 2 stop	0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

$\sim\text{Eprom} = A19 | A18 | A17$

$\sim\text{Sram 1} = A19 | A18 | \sim A17 | A16 | A15$

$\sim\text{Sram 2} = A19 | A18 | \sim A17 | A16 | \sim A15$

Interfețe seriale și paralele (circuiturile 8251 și 8255)



3. Interfetele de intrare/iesire ale microsistemului

Pentru comunicarea microprocesorului 8086 cu dispozitive externe, microsistemul utilizează două tipuri de interfețe de intrare/ieșire: o interfață serială și o interfață paralelă. Acestea sunt realizate cu ajutorul circuitelor **Intel 8251** și **Intel 8255**, care sunt conectate la magistrala de date și la magistrala de control a sistemului.

Selecția interfețelor se face prin intermediul unui **decodicator de adrese 74LS138**, care analizează liniile superioare ale magistralei de adrese și generează semnale de activare distincte pentru fiecare circuit. În acest mod, se asigură accesul exclusiv la un singur periferic la un moment dat și se evită conflictele de adresare.

Decodificatorul de adrese 74LS138 în sistem

Circuitul **74LS138** este un decodificator de tip 3-la-8, cu ieșiri active pe nivel logic LOW. Acesta este utilizat pentru decodificarea adreselor din spațiul de intrare/ieșire și pentru generarea semnalelor de selecție (chip-select) pentru interfețele periferice.

În cadrul microsistemului:

- intrările decodificatorului sunt conectate la liniile superioare de adresă ale microprocesorului;
- ieșirile decodificatorului sunt utilizate pentru activarea circuitelor 8251 și 8255;
- semnalele de enable ale decodificatorului sunt controlate astfel încât decodificarea să fie activă doar în cazul acceselor de tip I/O, indicate de semnalul **M/I $\bar{O}$** .

Prin utilizarea decodificatorului 74LS138, spațiul de adrese este împărțit logic, iar fiecare interfață răspunde doar la intervalul de adrese care îi este alocat.

Interfața serială – circuitul 8251

Circuitul **8251** este o interfață de comunicație serială programabilă, utilizată pentru transmiterea și recepția datelor în format serial. Acesta permite conversia datelor între format paralel (utilizat de microprocesor) și format serial (utilizat pe linia de comunicație).

8251 conține mai multe registre interne, care sunt accesate de microprocesor prin magistrala de date. Funcționarea circuitului este controlată prin registre de comandă și de mod, care stabilesc

parametri precum viteza de transmisie, numărul de biți de date sau tipul de paritate.

Pini și semnale importante ale circuitului 8251:

- **D0–D7** – magistrala de date, utilizată pentru transferul informațiilor între microprocesor și circuit;
- **$\overline{RD}$** – comandă de citire, utilizată pentru citirea datelor sau a stării circuitului;
- **$\overline{WR}$** – comandă de scriere, utilizată pentru încărcarea registrelor interne;
- **$\overline{CS}$** – semnal de selecție al circuitului, generat de decodificatorul de adrese;
- **C/D** – selectează între registrul de date și registrele de control;
- **RESET** – inițializează circuitul 8251;
- **TxD / RxD** – linii de transmisie și recepție serială;
- **TxC / RxC** – semnale de ceas pentru transmisie și recepție.

Prin intermediul acestor semnale, 8251 asigură comunicarea serială între microsistem și un dispozitiv extern, cum ar fi un terminal sau un calculator.

Interfața paralelă – circuitul 8255

Circuitul **8255** este o interfață paralelă programabilă, utilizată pentru conectarea dispozitivelor periferice care necesită transfer paralel de date. Acesta oferă o mare flexibilitate, permițând configurarea porturilor ca intrări sau ieșiri, în funcție de necesitățile aplicației.

8255 conține trei porturi paralele:

- **Port A** – port de 8 biți;

- **Port B** – port de 8 biți;
- **Port C** – port de 8 biți, care poate fi utilizat ca port complet sau împărțit în două jumătăți.

Pini și semnale importante ale circuitului 8255:

- **D0–D7** – magistrala de date pentru transferul informațiilor;
- **A0, A1** – selectează portul sau registrul de control accesat;
- **$\overline{RD}$** – comandă de citire;
- **$\overline{WR}$** – comandă de scriere;
- **$\overline{CS}$** – semnal de activare al circuitului, furnizat de decodificator;
- **RESET** – readuce circuitul într-o stare inițială;
- **PA0–PA7, PB0–PB7, PC0–PC7** – linii de intrare/ieșire pentru conectarea perifericelor.

Configurarea porturilor se realizează prin scrierea unui cuvânt de control în registrul de control al circuitului 8255, permițând adaptarea interfeței la diferite tipuri de dispozitive externe.

Funcționarea generală a interfețelor

În timpul unui acces de intrare/ieșire, microprocesorul generează adresa și semnalele de control corespunzătoare. Decodificatorul 74LS138 selectează circuitul periferic corespunzător, iar transferul de date se realizează prin magistrala de date, sub controlul semnalelor $\overline{RD}$ și $\overline{WR}$.

Prin utilizarea circuitelor 8251 și 8255, microsistemul beneficiază atât de comunicație serială, cât și de comunicație paralelă, asigurând o interfață completă și flexibilă cu mediul extern.

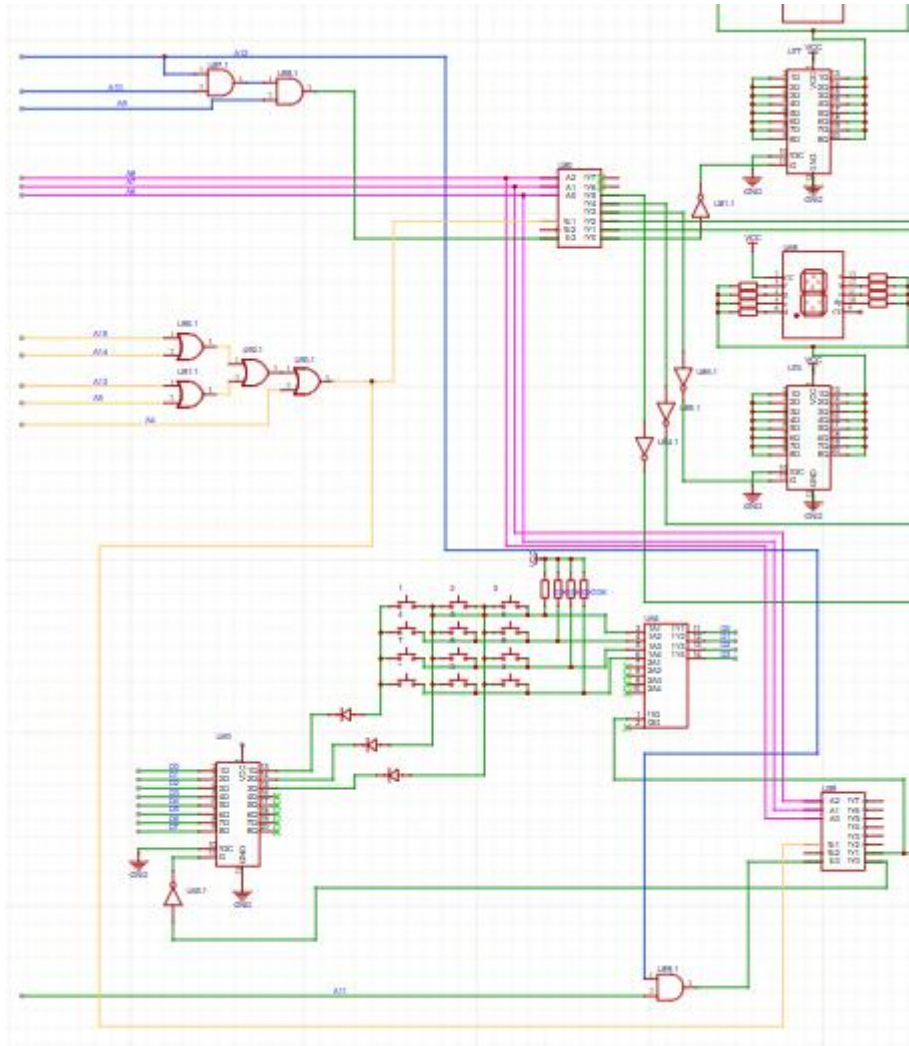
Tabel:

	...	adre sa	A1 5	A1 4	A1 3	A1 2	A1 1	A1 0	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	A1	A 0
S 1		0AF0 H	0	0	0	0	1	0	1	0	1	1	1	1	0	0	0	0
		0AF2 H	0	0	0	0	1	0	1	0	1	1	1	1	0	0	1	0
S 2		0BF0 H	0	0	0	0	1	0	1	1	1	1	1	1	0	0	0	0
		0BF2 H	0	0	0	0	1	0	1	1	1	1	1	1	0	0	1	0
P 1	Port A	0D70 H	0	0	0	0	1	1	0	1	0	1	1	1	0	0	0	0
	Port B	0D72 H	0	0	0	0	1	1	0	1	0	1	1	1	0	0	1	0
	Port C	0D74 H	0	0	0	0	1	1	0	1	0	1	1	1	0	1	0	0
	RCC	0D76 H	0	0	0	0	1	1	0	1	0	1	1	1	0	1	1	0
P 2	Port A	0C70 H	0	0	0	0	1	1	0	0	0	1	1	1	0	0	0	0
	Port B	0C72 H	0	0	0	0	1	1	0	0	0	1	1	1	0	0	1	0
	Port C	0C74 H	0	0	0	0	1	1	0	0	0	1	1	1	0	1	0	0
	RCC	0C76 H	0	0	0	0	1	1	0	0	0	1	1	1	0	1	1	0

$$\sim\text{SEL_0} = \text{A15} \mid \text{A14} \mid \text{A13} \mid \text{A12}$$

$$\sim\text{SEL_1} = \sim\text{A11} \mid \sim\text{A6} \mid \sim\text{A5} \mid \sim\text{A4}.$$

4. Decodificator de porturi



Decodificatorul de porturi – rol și funcționare

Pentru conectarea interfețelor de intrare/ieșire la microprocesorul 8086 este necesară o logică de selecție care să permită activarea unui singur periferic la un moment dat, în funcție de adresa generată de procesor. Această funcție este realizată cu ajutorul decodificatorului de porturi, implementat în proiect cu circuitul 74LS138.

Decodificatorul 74LS138 este un circuit de tip 3-la-8, care transformă o combinație de trei biți de adresă într-unul din opt semnale de ieșire, active pe nivel logic LOW. În cadrul microsistemului, aceste ieșiri sunt utilizate ca semnale de chip-select pentru interfețele de intrare/ieșire.

Utilizarea decodicatorului 74LS138 în spațiul I/O

În proiect, decodicatorul de porturi este conectat la liniile superioare ale magistralei de adrese, care definesc zona de intrare/ieșire accesată de microprocesor. Activarea decodicatorului este condiționată de semnalul $M/\overline{IO}$, astfel încât acesta să funcționeze exclusiv în timpul acceselor de tip I/O și să fie dezactivat în cazul acceselor la memorie.

Prin această metodă, se asigură separarea clară între:

- spațiul de memorie (EPROM, SRAM);
- spațiul de intrare/ieșire (interfețele 8251 și 8255).

Când microprocesorul execută o instrucțiune de intrare sau ieșire, liniile de adresă corespunzătoare sunt analizate de decodicator, iar una dintre ieșirile sale devine activă, selectând perifericul dorit.

Semnale și pini relevanți ai decodicatorului 74LS138

Circuitul 74LS138 dispune de:

- trei intrări de adresă (A_0, A_1, A_2), care determină ieșirea activă;
- trei intrări de activare (Enable), utilizate pentru controlul funcționării decodicatorului;
- opt ieșiri (Y_0-Y_7), active pe nivel LOW.

În schema microsistemului, intrările de activare sunt conectate astfel încât decodicatorul să fie activ doar în cazul acceselor de intrare/ieșire. Ieșirile sunt utilizate pentru a genera semnalele de selecție pentru circuitele 8251 și 8255, precum și pentru eventuale extensii ulterioare ale sistemului.

Faptul că ieșirile sunt active pe nivel LOW este avantajos, deoarece majoritatea circuitelor periferice utilizează semnale $\overline{CS}$ active pe zero, permițând o conectare directă, fără logică suplimentară complexă.

Am ales adrese random de la 1600 -> 1740 și de la 1900 -> 1980.

Tabel:

Adr	A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	Semna
1600	0	0	0	1	0	1	1	0	0	0	0	0	0	0	0	0	SA1
1640	0	0	0	1	0	1	1	0	0	1	0	0	0	0	0	0	SA2
1680	0	0	0	1	0	1	1	0	1	0	0	0	0	0	0	0	SA3
16C0	0	0	0	1	0	1	1	0	1	1	0	0	0	0	0	0	SA4
1700	0	0	0	1	0	1	1	1	0	0	0	0	0	0	0	0	SA5
1740	0	0	0	1	0	1	1	1	0	1	0	0	0	0	0	0	SA6
1900	0	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0	SL1
1940	0	0	0	1	1	0	0	1	0	1	0	0	0	0	0	0	ST1
1980	0	0	0	1	1	0	0	1	1	0	0	0	0	0	0	0	ST2

A11 -> always ,0' logic for DEC1

,1' logic for DEC2

A10 -> always ,1' logic for DEC1

,0' logic for DEC2

A9 -> always ,1' logic for DEC1

,0' logic for DEC2

A12 -> always ,1' logic

Decodificator 1:

$$\sim Ec1 = A4 \mid A5 \mid A13 \mid A14 \mid A15$$

$$Ec2 = A9 \& A10 \& A12$$

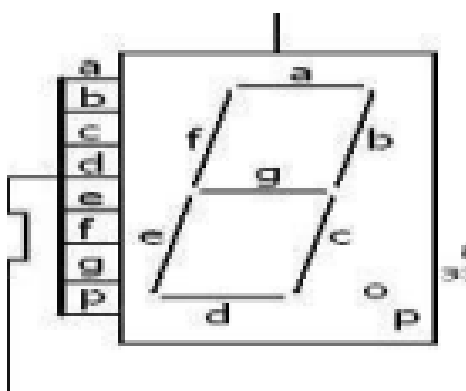
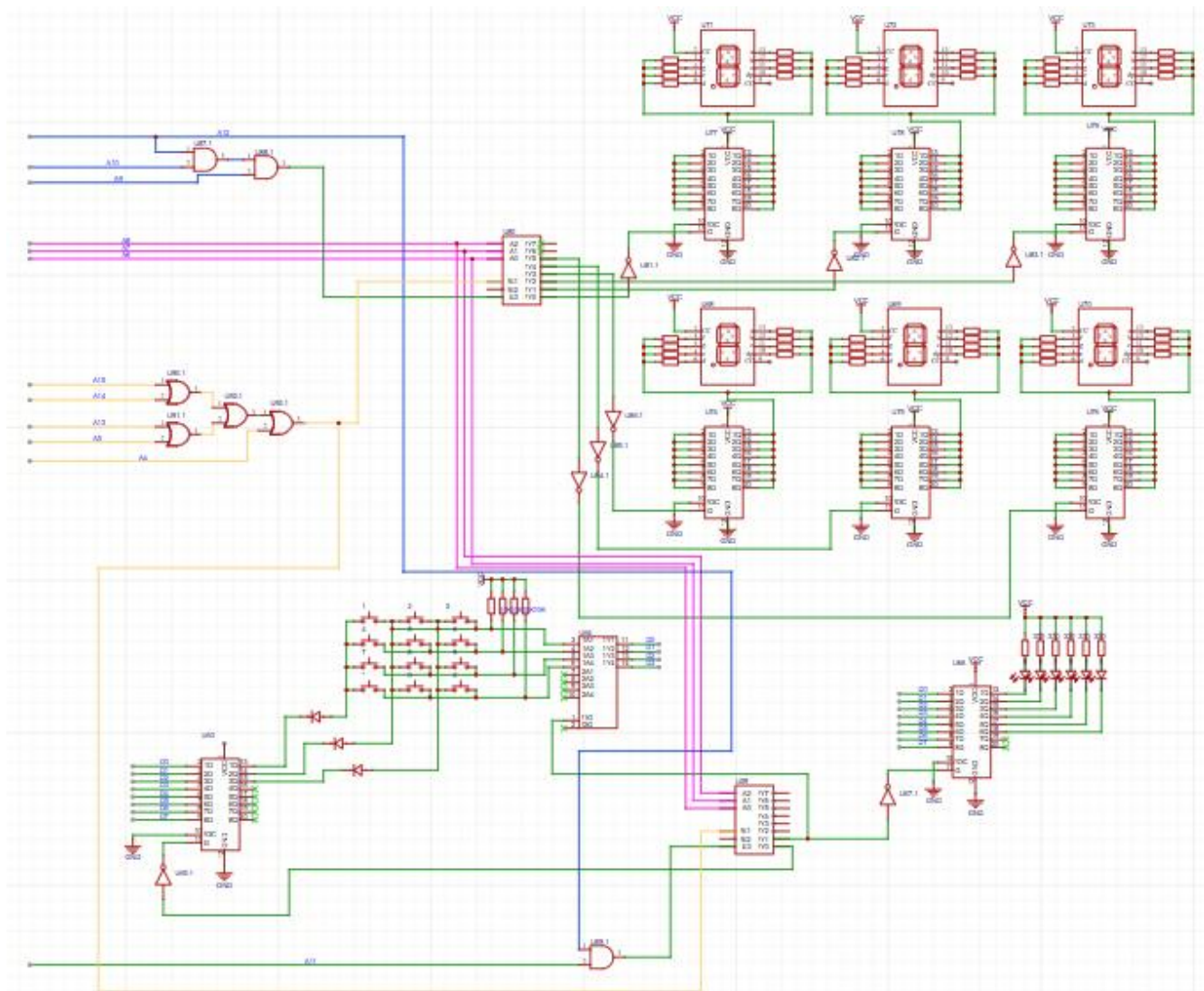
$$Ec3 = \sim A11$$

Decodificator 2:

$$Ec1 = A11 \& \sim A10 \& A9$$

$$Ec2 = A12$$

5. Conectarea afișajelor și a minitastaturii



Conectarea minitastaturii

Minitastatura este utilizată ca dispozitiv de **intrare**, permițând introducerea comenzilor sau a valorilor de către utilizator. Aceasta este organizată sub forma unei **matrici de butoane**, conectată la sistem prin circuite de tip buffer și latch.

Liniile tastaturii sunt menținute într-o stare logică definită cu ajutorul rezistențelor de pull-up, astfel încât apăsarea unei taste să producă o modificare clară a nivelului logic. Semnalele provenite de la tastatură sunt citite de microprocesor prin magistrala de date, în urma unui acces de tip I/O.

Selecția minitastaturii se face tot prin decodificatorul **74LS138**, care activează circuitul de intrare doar atunci când este accesat portul corespunzător. În acest mod, tastatura nu interferează cu celelalte dispozitive de intrare/ieșire din sistem.

Citirea tastelor se realizează prin scanare, microprocesorul verificând succesiv starea liniilor de intrare pentru a identifica tasta apăsată. Această metodă permite utilizarea unui număr redus de linii de conexiune, păstrând în același timp funcționalitatea completă a tastaturii.

	Coloana 1	Coloana 2	Coloana 3
Linia 1	1	2	3
Linia 2	4	5	6
Linia 3	7	8	9
Linia 4	*	0	#

	Binar	Hexa	Taste detectate
Linia 1	1111 1110	FE	1,2,3
Linia 2	1111 1101	FD	4,5,6
Linia 3	1111 1011	FB	7,8,9
Linia 4	1111 0111	F7	*,0,#

Conectarea LED-urilor în microsistem

Pentru afișarea informațiilor binare și a stărilor interne ale sistemului, microsistemul utilizează un grup de LED-uri, conectate ca dispozitiv de ieșire. LED-urile sunt comandate prin intermediul magistralei de date, valorile

transmise de microprocesor fiind memorate într-un circuit de tip latch (74LS373).

Latch-ul are rolul de a păstra stabilă valoarea ieșirilor chiar și după terminarea ciclului de scriere, lucru necesar deoarece microprocesorul utilizează magistrala de date și pentru alte operații. Intrările latch-ului sunt conectate la liniile de date, iar ieșirile sale comandă LED-urile prin rezistențe de limitare a curentului.

Activarea latch-ului se face printr-un semnal de selecție generat de decodificatorul de porturi 74LS138, astfel încât LED-urile să fie actualizate doar atunci când microprocesorul accesează portul corespunzător. Scrierea efectivă este controlată de semnalul $\overline{WR}$, iar LED-urile reflectă starea logică a fiecărui bit scris.

Ledul	Bitul	Valoarea Binară	Valoarea în Hexa
LED1	D0	1111 1110	FE
LED2	D1	1111 1101	FD
LED3	D2	1111 1011	FB
LED4	D3	1111 0111	F7
LED5	D4	1110 1111	EF
LED6	D5	1101 1111	DF

Modulul de afișare cu 7 segmente

Pentru afișarea informațiilor numerice, microsistemul utilizează un modul de afișare cu 7 segmente, alcătuit din mai multe afișaje identice. Fiecare afișaj este comandat prin intermediul unui latch dedicat, care memorează codul segmentelor ce trebuie aprinse.

Liniile de date ale microprocesorului sunt conectate la intrările latch-urilor, iar ieșirile acestora sunt legate la segmentele afișajului (a–g și punctul zecimal). Rezistențele inserate pe fiecare segment au rolul de a limita curentul și de a proteja LED-urile din afișaj.

Selecția fiecărui afișaj se realizează prin semnale distincte generate de 74LS138, astfel încât fiecare cifră să poată fi comandată individual. Această metodă permite afișarea de valori diferite pe fiecare poziție, utilizând aceeași magistrală de date.

Semnalul de scriere ($\overline{WR}$) determină momentul în care valoarea de pe magistrala de date este memorată în latch, iar afișajul păstrează informația până

la următoarea actualizare. Prin această organizare se obține o afișare stabilă și clară a datelor numerice.

Digit în hexa	Segmentul pornit	În Binar	Hexa
0	a b c d e f	1100 0000	C0
1	b c	1111 1001	F9
2	a b d e g	1010 0100	A4
3	a b c d g	1011 0000	B0
4	b c f g	1001 1001	99
5	a c d f g	1001 0010	92
6	a c d e f g	1000 0010	82
7	a b c	1111 1000	F8
8	a b c d e f g	1000 0000	80
9	a b c d f g	1001 0000	90
A	a b c e f g	1000 1000	88
B	c d e f g	1000 0011	83
C	a d e f	1100 0110	C6
D	b c d e g	1010 0001	A1
E	a d e f g	1000 0110	86
F	a e f g	1000 1110	8E

Dacă 1 e aprins, toate sunt 1 în afară de b c, dp-g-f-e-d-c-b-a, dp = 1.

6. Rutinele:

; Rutine de inițializare interfață serială 8251

UART_DATA EQU 0AF0H ; adresa registrului de date al 8251

UART_CTRL EQU 0AF2H ; adresa registrului de control/comandă

INIT_8251:

MOV AL, 40H ; încărcăm valoarea de reset software în AL

OUT UART_CTRL, AL ; trimitem resetul către 8251

MOV AL, 4EH ; setăm modul: asincron, 8 biți, 1 stop, fără paritate
OUT UART_CTRL, AL ; scriem modul de funcționare în 8251

MOV AL, 05H ; activăm transmisia și recepția
OUT UART_CTRL, AL ; trimitem comanda de pornire
RET ; revenim din rutină

asm

; Rutine de inițializare interfață paralelă 8255

PPI_CTRL EQU 0D76H ; adresa registrului de control al 8255

INIT_8255:

MOV AL, 80H ; configurăm porturile (A ieșire, B intrare, C ieșire)
OUT PPI_CTRL, AL ; trimitem cuvântul de control către 8255
RET ; revenim din rutină

asm

; Rutina de emisie caracter pe interfața serială 8251

SERIAL_TX:

WAIT_TX:

IN AL, UART_CTRL ; citim registrul de stare al 8251
TEST AL, 01H ; verificăm bitul TxRDY (gata de transmisie)
JZ WAIT_TX ; dacă nu e gata, așteptăm

```
MOV AL, CL          ; mutăm caracterul de transmis din CL în AL
OUT UART_DATA, AL   ; trimitem caracterul pe linia serială
RET                 ; revenim din rutină
```

asm

; Rutina de recepție caracter pe interfața serială 8251

SERIAL_RX:

WAIT_RX:

```
IN AL, UART_CTRL    ; citim registrul de stare
TEST AL, 02H         ; verificăm bitul RxRDY (date disponibile)
JZ WAIT_RX          ; dacă nu sunt date, așteptăm
```

```
IN AL, UART_DATA     ; citim caracterul recepționat
MOV CL, AL           ; salvăm caracterul în CL
RET                 ; revenim din rutină
```

asm

; Rutina de emisie caracter pe interfața paralelă 8255

PPI_PORTA EQU 0D70H ; adresa portului A al 8255

PARALLEL_OUT:

```
MOV AL, CL          ; mutăm caracterul de transmis în AL
OUT PPI_PORTA, AL   ; scriem caracterul pe portul paralel
RET                 ; revenim din rutină
```

asm

; Rutine pentru controlul LED-urilor

LED_PORT EQU 2800H ; adresa portului pentru LED-uri

LED_SET:

OUT LED_PORT, AL ; scriem masca de LED-uri (0 aprinde LED-ul)

RET ; revenim din rutină

LED_CLEAR:

MOV AL, 0FFH ; setăm toți biții la 1 (toate LED-urile stinse)

OUT LED_PORT, AL ; trimitem valoarea pe port

RET ; revenim din rutină

asm

; Rutina de afișare caracter hexazecimal pe afișaj cu 7 segmente

SEG_PORT EQU SA3 ; adresa portului pentru afișajul cu 7 segmente

SEG_TABLE DB 0C0H,0F9H,0A4H,0B0H ; coduri pentru cifrele 0–3

DB 099H,092H,082H,0F8H ; coduri pentru cifrele 4–7

DB 080H,090H,088H,083H ; coduri pentru cifrele 8–B

DB 0C6H,0A1H,086H,08EH ; coduri pentru cifrele C–F

SHOW_HEX:

MOV BX, OFFSET SEG_TABLE ; încărcăm adresa tabelului de coduri

XLAT ; înlocuim AL cu codul de afișare corespunzător

OUT SEG_PORT, AL ; trimitem codul către afișaj

RET ; revenim din rutină

asm

; Rutina de scanare a minitastaturii

ROW_PORT EQU 2880H ; port pentru selectarea rândurilor

COL_PORT EQU 2840H ; port pentru citirea coloanelor

SCAN_KEY:

MOV AL, 0FEH ; activăm rândul 1 (0 logic)

OUT ROW_PORT, AL ; trimitem selecția rândului

IN AL, COL_PORT ; citim coloanele

CMP AL, 0FFH ; verificăm dacă vreo tastă este apăsată

JNE KEY_FOUND ; dacă da, ieșim

MOV AL, 0FDH ; activăm rândul 2

OUT ROW_PORT, AL

IN AL, COL_PORT

CMP AL, 0FFH

JNE KEY_FOUND

MOV AL, 0FBH ; activăm rândul 3

OUT ROW_PORT, AL

IN AL, COL_PORT

CMP AL, 0FFH

JNE KEY_FOUND

MOV AL, 0F7H ; activăm rândul 4

OUT ROW_PORT, AL

IN AL, COL_PORT

CMP AL, 0FFH

JNE KEY_FOUND

JMP SCAN_KEY ; reluăm scanarea dacă nu e nicio tastă

KEY_FOUND:

RET ; revenim când o tastă este detectată

asm

; Rutina de întârziere pentru debouncing tastatură

KBD_DELAY:

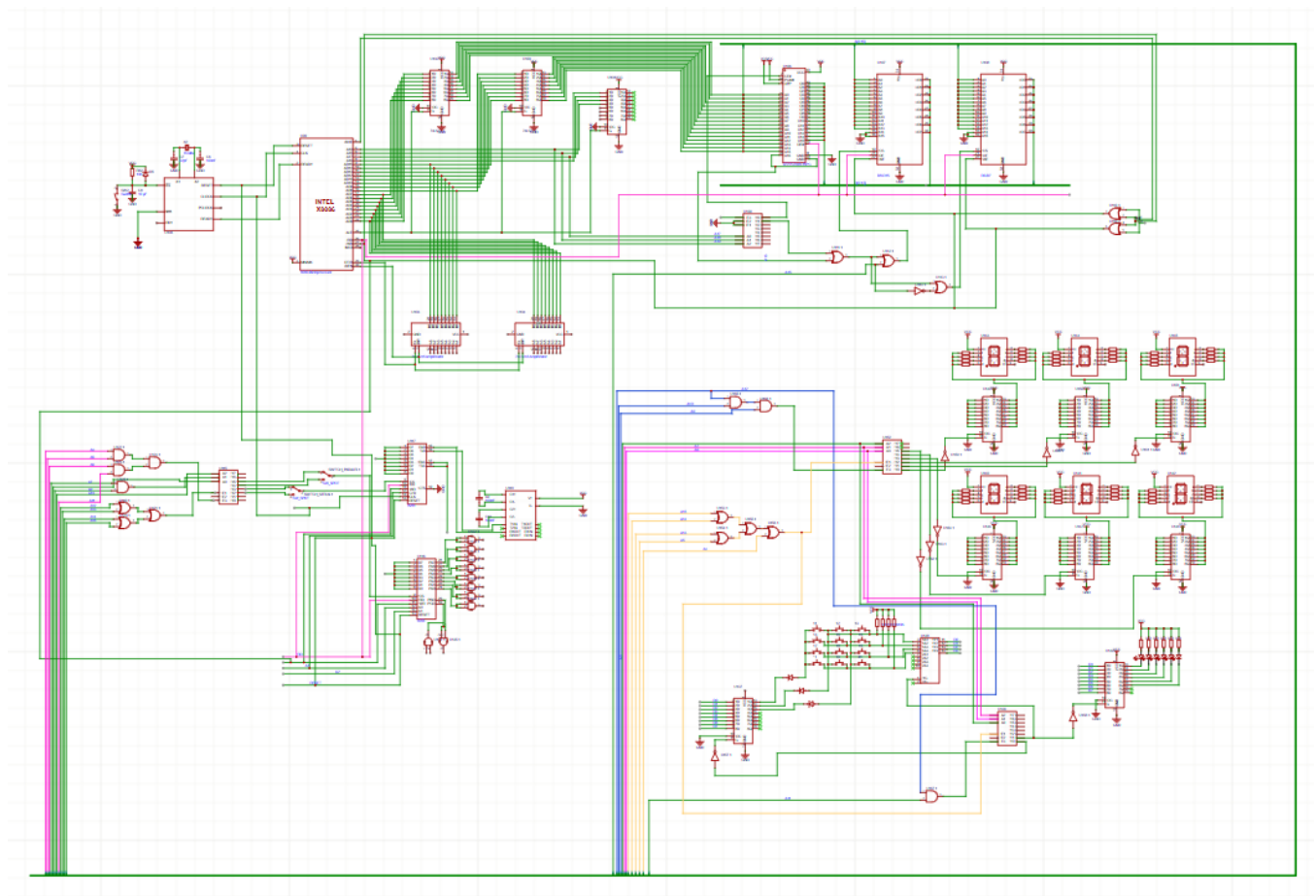
MOV CX, 5000 ; încărcăm un contor de întârziere

DLY:

LOOP DLY ; scădem contorul până ajunge la zero

RET ; revenim din rutină

Schema Completă:



Bibliografie:

http://tet.pub.ro/Documente/Materiale%20pentru%20studen%C8%9Bi/An%20III/Microprocesoare2/MP_CAP_11.pdf

[http://www.calc.fcim.utm.md/biblioteca/arhiva/Anul%20II/Semestru%20II/Arhitectura%20Calculatoarelor/Indrumar%20Proiect%20UCro\(fr\).pdf](http://www.calc.fcim.utm.md/biblioteca/arhiva/Anul%20II/Semestru%20II/Arhitectura%20Calculatoarelor/Indrumar%20Proiect%20UCro(fr).pdf)

Mircea Popa, „Proiectarea microsistemelor digitale”, Editura Orizonturi Universitare, Timisoara, 2003